

cloudbtorch / develop — Step 6 how-to

Running & using the cube inversion `fit_cube.py`: the step-1–5 stack (fitted PCA background $\oplus$ two absorbing clouds, fit by a Fourier neural field with explicit regularisation) as one configurable, resumable CUDA script. With the baked-in defaults it reproduces the tuned step-5 notebook; the full cube is reached by config alone.

1. ENVIRONMENT

Python + CUDA torch: `/home/milic/miniconda3/envs/pt/bin/python` (torch 2.11, CUDA). No new physics — it reuses `pca_background`, `pinn_model`, `composite_model`, `regularizers` unchanged. The data path auto-detects from the `data.path` list; add yours there or via `--set`.

2. QUICK START

```
PY=/home/milic/miniconda3/envs/pt/bin/python
cd /home/milic/codes/cloudbtorch/develop
$PY fit_cube.py --out run1 --report          # defaults == step-5 notebook
$PY fit_cube.py --dump-config cfg.json       # write full default config to edit
$PY fit_cube.py --config cfg.json --out run2  # run an edited config
```

3. CONFIGURING (THREE WAYS; EVERYTHING LIVES IN THE CONFIG)

(1) edit a file: `--dump-config cfg.json`, edit, `--config cfg.json`. (2) inline: `--set data.crop=32 train.n_iters=2000` (values parse as JSON: `null`, `true`, numbers, `[0,0]`). (3) flags: `--out`, `--device`. Precedence: defaults $\rightarrow$ `--config` $\rightarrow$ `--set` $\rightarrow$ flags; the *resolved* config (with the scaled σ) is written to `DIR/config_used.json`. Key knobs: `data.crop/data.n_t` (`null` = full field / all frames), `train.batch` (`null` = full batch), `train.n_iters`, and the whole reg block (smoothness w_{xy}/w_t , hinges, `no_emission`, and the anchor weight — the main background lever). The Fourier bandwidths **scale with the domain automatically** (`sigma_xy_ref/sigma_t_ref` at `crop_ref/n_t_ref`), so the tuning transfers as the crop/frames grow — no manual σ retune.

4. OUTPUTS (IN THE `--OUT DIR`)

`params.npz` — `c` (`Nt,Cy,Cx,6`), `p_cloud` (`Nt,Cy,Cx,10`) (order in `cloud_names`, incl. frozen loga), plus `wl`, `lam`, `Iqs`, `ts`, `ys`, `xs`, `cfg.checkpoint.pt` — self-contained (field + optimiser + scheduler + RNG + config + PCA basis), written every `io.ckpt_every` iters and at the end. `config_used.json`; with `--report`: `loss` / `obs_vs_fit` / `emission_hist` / `spectra` PNGs. Console [metrics]: RMS (% of mean I) and the background emission diagnostic.

5. RESUME / EVALUATE-ONLY

```
$PY fit_cube.py --resume run1/checkpoint.pt          # continue to n_iters
$PY fit_cube.py --resume run1/checkpoint.pt --no-train --report  # eval + figures
```

Resume restores everything and continues the cosine schedule to the original `n_iters`. (To train *longer*, start a fresh run with a larger `train.n_iters`.)

6. SCALE TO THE FULL CUBE

```
$PY fit_cube.py --set data.crop=null data.n_t=null train.batch=8192 \
                  train.n_iters=30000 --out full --report
```

`crop=null/n_t=null` take the whole field / all frames; σ rescales itself; pick `train.batch` to fit GPU memory (the regulariser is mesh-free, so minibatching is exact). Then **tune** reg at that scale — the weights are relative to the χ^2 term.

7. USING THE RESULTS

```
import numpy as np
d = np.load("run1/params.npz", allow_pickle=True)
vlos1 = d["p_cloud"][..., 2]  # (Nt,Cy,Cx) LOS velocity, cloud 1, km/s
S1     = d["p_cloud"][..., 0]  # dtau1=[...,1], dv1=[...,3]; names in d["cloud_names"]
```

`p_cloud` gives the physical cloud parameters directly (`c` is the 6 background PCA coeffs). To *re-synthesise* spectra or the background, the checkpoint is self-contained — reload it (it carries the PCA basis + field weights) and call the same forward, or run `--resume ... --no-train --report`.

Companion to `step_6_outline.pdf` (the plan). Code: `fit_cube.py`; guide: `STEP6_HOWTO.md`. The step-1–5 modules are reused unchanged.